# Furuno DRS4D-NXT Radar

## Wire Protocol & System Technical Manual

J.C. Vaught

March 2026

A complete technical reference for the Furuno DRS4D-NXT marine radar system, covering the wire protocol, echo data encoding, system architecture, web interface, and development guide.

# Contents

**I Overview & Usage** **2**

**1 Introduction** **2**
1.1 Hardware Overview . . . 2
1.2 Software Components . . . 2
1.3 Reverse Engineering Context . . . 3

**2 Quick Start & Installation** **4**
2.1 Prerequisites . . . 4
2.2 Building . . . 4
2.3 First Run . . . 4
2.4 Auto-Discovery . . . 4

**3 Web Interface** **6**
3.1 Starting the Server . . . 6
3.2 Page Tour . . . 6
3.2.1 Controls (index.html) . . . 6
3.2.2 Radar View (radar.html) . . . 6
3.2.3 Settings (settings.html) . . . 6
3.2.4 System Status (status.html) . . . 7
3.3 PPI Display . . . 7

**4 Recording & Data Formats** **9**
4.1 Starting and Stopping Recording . . . 9
4.2 Output Directory Structure . . . 9
4.3 CSV Format . . . 9
4.4 Settings Snapshot . . . 10

**5 Pattern System** **11**
5.1 What Patterns Do . . . 11
5.2 Simple Pattern Format (7-Field) . . . 11
5.3 Advanced Pattern Format (13-Field) . . . 11
5.4 Pattern Queue . . . 11
5.5 Loading Patterns . . . 12
5.6 NaN Override Behavior . . . 12

**II Software Internals** **13**

**6 System Architecture Overview** **13**
6.1 Module Summary . . . 13
6.2 System-Level Block Diagram . . . 13
6.3 Module-Level Diagram . . . 13
6.4 Data Flow Diagram . . . 13
6.5 Concurrency Model . . . 13

**7 PPI Rendering** **14**
7.1 Coordinate Transformation . . . 14
7.2 Color Mapping . . . 14
7.3 Overlay Elements . . . 14
7.4 PNG Encoding . . . 14
7.5 How to Modify . . . 14

**8 Web Server** **15**
8.1 Framework . . . 15
8.2 Shared State . . . 15
8.3 Route Registration . . . 15
8.4 Handler Pattern . . . 15
8.5 Echo Listener Thread . . . 15
8.6 Static File Serving . . . 15
8.7 How to Add a New Endpoint . . . 15

**9 REST API Reference** **16**
9.1 Power and Connection . . . 16
9.2 Settings . . . 16
9.3 State Queries . . . 16
9.4 Echo Image . . . 16
9.5 Recording . . . 16
9.6 Pattern Execution . . . 16
9.7 Stub Endpoints . . . 16

**10 Recording System Internals** **17**
10.1 Echo Listener Integration . . . 17
10.2 Directory Creation . . . 17
10.3 CSV Writing . . . 17
10.4 Settings Snapshots . . . 17
10.5 Setting Change History . . . 17

**11 Pattern Execution Engine** **18**
11.1 Step Parsing . . . 18
11.2 Differential Command Generation . . . 18
11.3 Single Pattern Execution . . . 18
11.4 Queue Execution . . . 18
11.5 NaN Override Logic . . . 18
11.6 Stop Mechanism . . . 18

**12 CLI Modes (All Other Modes)** **19**
12.1 Handshake . . . 19
12.2 TX On / TX Off . . . 19
12.3 Full Cycle . . . 19
12.4 Render Live . . . 19
12.5 Echo Listen . . . 19
12.6 Command Test . . . 19
12.7 Dump Raw . . . 19

12.8 Test Commands . . . 19
12.9 Validate . . . 19

**13 Connection State Machine** **20**
13.1 States and Transitions . . . 20
13.2 RadarError Enum . . . 20
13.3 Timeout Values . . . 20
13.4 Graceful Degradation . . . 20
13.5 Keepalive Handling . . . 20

**14 Testing & Validation** **21**
14.1 Running Tests . . . 21
14.2 Test Categories . . . 21
14.2.1 Connection Tests . . . 21
14.2.2 TX Tests . . . 21
14.2.3 Settings Tests . . . 21
14.2.4 Echo Impact Tests . . . 21
14.2.5 Render Integration Test . . . 21
14.3 A/B Command Testing . . . 21
14.4 Validation Suite . . . 21

**III Echo Data Pipeline** **22**

**15 Echo Data Format** **22**
15.1 Packet Structure . . . 22
15.2 Header Field Breakdown . . . 22
15.3 Echo Metadata Extraction . . . 22
15.4 Spoke Sub-Header . . . 22
15.5 Multi-Spoke Packing . . . 22
15.6 Azimuth and Heading . . . 22
15.7 Typical Packet Characteristics . . . 22

**16 Echo Encoding Types** **23**
16.1 Encoding 0: Raw Uncompressed . . . 23
16.2 Encoding 1: RLE with Repeat Marker . . . 23
16.3 Encoding 2: Delta RLE with Previous Spoke Reference . . . 23
16.4 Encoding 3: 2-Bit RLE with Dual Reference . . . 23
16.5 Encoding Selection . . . 23
16.6 Display Value Conversion . . . 23
16.7 Rotation Detection . . . 23

**17 Doppler Processing** **24**
17.1 Status Field Encoding . . . 24
17.2 Echo Byte Encoding . . . 24
17.3 Downstream Usage . . . 24
17.4 Practical Notes . . . 24

**18 Dual-Pulse Interleaving** **25**

18.1 Pulse Types . . . 25
18.2 Range Dependence . . . 25
18.3 Rez Boost Effect . . . 25
18.4 Spoke Handling . . . 25

**IV Network & Wire Protocol 26**

**19 Network Architecture 26**
19.1 Topology . . . 26
19.2 Port Map . . . 26
19.3 Auto-Discovery and Network Configuration . . . 26
19.3.1 Platform-Specific IP Alias . . . 26

**20 Control Protocol 27**
20.1 Message Format . . . 27
20.2 Binary Keepalive . . . 27
20.3 Binary vs ASCII Disambiguation . . . 27
20.4 Response Types . . . 27
20.5 Response Parsing Pipeline . . . 27
20.6 Command Acknowledgment . . . 27
20.7 Control Channel Drain . . . 27
20.8 Initialization Sequence . . . 27
20.8.1 Phase 1: Discovery . . . 27
20.8.2 Phase 2: Authentication . . . 27
20.8.3 Phase 3: Control Channel . . . 27
20.8.4 Phase 4: Firmware Query . . . 27
20.8.5 Phase 5: Register Dump . . . 27
20.8.6 Phase 6: Steady State . . . 27

**21 Command Reference 28**
21.1 TX State (`$S69`) . . . 28
21.2 Range (`$S62`) . . . 28
21.3 Gain (`$S63`) . . . 28
21.4 Sea Clutter (`$S64`) . . . 28
21.5 Rain Clutter (`$S65`) . . . 28
21.6 Pulse Width (`$S68`) . . . 28
21.7 STC Near (`$S85`) . . . 28
21.8 STC Middle (`$S86`) . . . 28
21.9 STC Far (`$S87`) . . . 28
21.10 Main Bang Suppression (`$S83`) . . . 28
21.11 Antenna Height (`$S84`) . . . 28
21.12 Antenna Revolution (`$S89`) . . . 28
21.13 Interference Rejection (`$S67`, param[2]=2) . . . 28
21.14 Noise Reduction (`$S67`, param[2]=3) . . . 28
21.15 Rez Boost (`$SEE`) . . . 28
21.16 Bird Mode (`$SED`) . . . 28
21.17 Doppler / Target Analyzer (`$SEF`) . . . 28

21.18No-Transmit Sectors (`$S77`) . . . . . 28
21.19Query Commands . . . . . 28

**22 Periodic Telemetry** **29**
22.1 Telemetry Message Table . . . . . 29
22.2 Telemetry Processing . . . . . 29
22.3 Parsed vs Discarded Data . . . . . 29

**23 Proprietary Library Reference** **30**
23.1 Module Breakdown . . . . . 30
23.2 Shared Command IDs . . . . . 30

# Part I

# Overview & Usage

## 1 Introduction

This document is the complete technical reference for the Furuno DRS4D-NXT marine radar system as interfaced through the `furuno-server` Rust application. It covers radar operation via a browser-based web interface, the internal software architecture, echo data encoding and decoding, and the full wire protocol. The intended audiences are new users learning to operate the system, contributors modifying the codebase, and developers building their own applications on top of the radar hardware.

### 1.1 Hardware Overview

The Furuno DRS4D-NXT is a solid-state Doppler marine radar operating in the X-band at 9.41 GHz. The scanner unit contains the antenna, transmitter, and receiver in a single sealed radome enclosure. It connects to a host processor—in this deployment, a Raspberry Pi—via a standard Ethernet cable. The scanner is assigned a fixed IP address of `172.31.3.226` on a `172.31.0.0/16` subnet.

The radar transmits at up to 25 watts using solid-state amplification rather than a traditional magnetron, which eliminates warm-up time and provides consistent performance across the operating temperature range. The antenna rotates at either 24 or 48 RPM depending on the selected speed mode. Echo data is delivered as radial spokes, with 8192 unique azimuth positions per full rotation and up to 868 range bins per spoke. The system supports six pulse width settings (S1, S2, M1, M2, M3, L), 18 range scales from 1/16 NM to 48 NM, and built-in Doppler target discrimination.

### 1.2 Software Components

The system consists of three software components.

*furuno-server* is a Rust binary that implements the complete Furuno wire protocol from scratch. It handles network discovery, authentication, control commands, echo data reception and decoding across four encoding types, PPI (Plan Position Indicator) rendering, and serves a web interface for browser-based operation. The application is built with Axum 0.7 for the HTTP server and Tokio for the async runtime. All radar communication—TCP control, UDP multicast echo reception, and subnet management—is handled natively without any Furuno proprietary libraries.

The *web interface* is a set of static HTML, CSS, and JavaScript files served by the Rust application. It provides four pages: a controls dashboard for quick radar operation, a full-screen PPI display with history compositing, a detailed settings panel, and a system status view. The interface communicates with the server through a REST API and polls for updated radar images at 500 ms intervals.

The *documentation* consists of this technical manual and inline source code comments. The manual is maintained as a single self-contained LaTeX file with all diagrams rendered via TikZ.

### 1.3 Reverse Engineering Context

All protocol knowledge in this document was obtained through reverse engineering. The primary source was the proprietary Furuno shared library `libNAVNETDLL.so`, which was analyzed through symbol table inspection, function call tracing, and behavioral observation. Live packet captures between the library and the radar hardware provided ground truth for the authentication handshake, control message format, and echo data encoding. The Rust implementation in `furuno-server` was built incrementally by reproducing each protocol phase and validating against the radar's actual responses.

The original C++ application that used `libNAVNETDLL.so` directly served as a reference implementation during this process. Command IDs, parameter ranges, and encoding schemes documented here have been verified against both the disassembled library and live radar behavior. Where the proprietary library's behavior was ambiguous or undocumented, the values in this manual reflect what was observed to work reliably in practice.

# 2 Quick Start & Installation

## 2.1 Prerequisites

The Rust toolchain is required to build the application. Install it via `rustup` if it is not already present.

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
```

The application builds and runs on Linux (x86_64 and aarch64), macOS, and Windows. The primary deployment target is a Raspberry Pi running Debian, connected to the radar via its Ethernet port. No external C libraries or Furuno SDKs are required.

## 2.2 Building

From the repository root, build the release binary.

```
cd furuno-server
cargo build --release
```

The compiled binary is located at `furuno-server/target/release/furuno-server`. A release build is recommended for deployment as it enables compiler optimizations that significantly improve echo decoding and PPI rendering throughput.

## 2.3 First Run

Start the web server with the `web` mode.

```
cargo run --release -- web
```

The web interface is then available at `http://localhost:8080` in any browser. On startup, the application attempts to discover the radar automatically (see below). If the radar is not reachable, the web server starts anyway and serves a degraded interface. The `/reconnect` endpoint can be used to retry the connection later.

## 2.4 Auto-Discovery

The application resolves the radar and host IP addresses through a three-stage fallback process.

First, it checks for explicit command-line flags. If `--radar-ip` is provided, that address is used directly. The `--host-ip` flag is optional; if omitted, the application determines a suitable host address on the radar's subnet automatically.

```
cargo run --release -- --radar-ip 172.31.3.226 web
```

Second, if no flags are provided, the application broadcasts UDP discovery beacons on port 10010 across all detected network interfaces and listens for 3 seconds. If the radar responds, both the radar and host addresses are extracted from the response.

Third, if discovery fails, the application falls back to the default addresses: `172.31.3.226` for the radar and `172.31.1.100` for the host. If the host does not already have an interface on the `172.31.0.0/16` subnet, a secondary IP alias is added to the physical Ethernet interface automatically. On Linux this uses `ip addr add`; on macOS, `ifconfig alias`.

# 3 Web Interface

## 3.1 Starting the Server

The web server starts when the application is launched in `web` mode (see §**??**). By default it listens on port 8080 and serves static files from the `../web` directory relative to the binary. Both values can be overridden with positional arguments. The server starts even if the radar is not reachable, allowing the interface to load and wait for a connection. The `/reconnect` API endpoint triggers a fresh authentication and initialization attempt without restarting the server.

## 3.2 Page Tour

The web interface consists of four HTML pages, all sharing a common header with navigation links, a consistent CSS stylesheet, and a single shared JavaScript file (`script.js`).

### 3.2.1 Controls (index.html)

The controls page is the default landing page and provides the most commonly used radar operations. It is divided into two panels. The left panel displays a radar preview image (refreshed every 2 seconds) with a link to the full-screen radar view. The right panel contains quick controls for TX power (on/off), CSV recording (start/stop), range selection (17 scales from 0.0625 to 36 NM), pulse width (S1 through L), manual gain (0–100), and RezBoost level (off/low/medium/high). Each control has an apply button and a refresh button that queries the radar's current state.

Below the quick controls is a pattern loader panel. Users can load a pattern file (`.txt` or `.csv`), set a repeat count ($-1$ for infinite looping), assign an optional label, and add the pattern to a queue. The queue is displayed as an ordered list, and patterns execute sequentially when the RUN button is pressed. A STOP button halts execution mid-pattern, and RESET clears the queue.

### 3.2.2 Radar View (radar.html)

The radar view page provides a full-screen PPI display. The primary display element is an HTML5 canvas (1024×1024 pixels) that composites multiple frames of radar data to show target history. Below the canvas are controls for track length (1–50 frames), current frame color, and track history color, with a reset button to restore defaults. An `<img>` element behind the canvas serves as a fallback if canvas rendering is unavailable. The canvas is refreshed every 500 ms by fetching the latest PPI image from the `/newest_png` endpoint.

### 3.2.3 Settings (settings.html)

The settings page provides granular control over all radar parameters. It is organized into panels grouped by function.

The *connection panel* shows the current connection status and provides reconnect and refresh buttons. A small radar preview image is displayed alongside.

The *power control panel* provides TX on/off buttons and a refresh button to query the current transmitter state.

The *range and pulse panel* allows independent or combined adjustment of range and pulse width, with the same options available on the controls page.

The *gain panel* supports both manual and automatic gain modes. In manual mode, the user sets a value from 0 to 100. In auto mode, a bias offset (0–100) adjusts how aggressively the automatic gain control responds.

The *STC panel* exposes three sensitivity time control zones—near, mid, and far—each with an independent numeric input.

The *signal processing panel* groups main bang suppression (0–255), interference rejection (on/off), bird mode (off/low/medium/high), and Doppler target analysis (off, internal heading, external heading).

The *clutter panel* provides sea clutter (A/C SEA) and rain clutter (A/C RAIN) controls, each with manual and automatic modes. Manual mode sets a fixed rejection level; auto mode adjusts a bias for the radar's internal adaptive filter.

The *antenna speed panel* controls rotation speed (auto/low/high) and a high-RPM override toggle.

The *RezBoost and display panel* sets the resolution enhancement level and display parameters including sweep line count and interpolation mode.

The *pattern loader* on this page has the same functionality as the one on the controls page, with the addition of a manual text input option for entering pattern steps directly.

#### 3.2.4 System Status (status.html)

The system status page displays live telemetry from both the radar and the host system, updated every 500 ms.

The *radar connection* panel shows connection status, radar IP, host IP, TX state, and firmware version. The *session* panel shows server uptime, total rotations received, recording state, the current recording directory, and pattern queue status. The *active settings* panel displays a read-only summary of all current radar parameters: range, pulse, gain, RezBoost, antenna speed, Doppler, interference rejection, bird mode, main bang suppression, STC values, and clutter settings.

The *host resources* panels display CPU load (overall and per-core), memory usage, storage usage, network throughput, and CPU temperature. Each metric includes a progress bar and numeric label.

### 3.3 PPI Display

The PPI (Plan Position Indicator) display renders radar echo data as a top-down circular image with the radar at the center. The server renders each complete antenna rotation into a 640×640 pixel PNG image using the warm-tinted color mapping described in §7.2. The browser fetches the latest image via the `/newest_png` endpoint.

On the radar view page, the JavaScript composites multiple frames onto an HTML5 canvas to show

target motion history. The *track length* slider controls how many previous frames are retained (1–50). The *current color* picker tints the most recent frame, and the *track color* picker tints older frames. Older frames are drawn with decreasing opacity to produce a fading trail effect. This compositing runs entirely in the browser; the server delivers only single-frame PNG images.

# 4 Recording & Data Formats

## 4.1 Starting and Stopping Recording

Recording is toggled through the web interface or by calling the `/toggle_recording` API endpoint with a `state` parameter of `start` or `stop`. When recording starts, the server creates a new timestamped directory under `recordings/` with `csv/` and `png/` subdirectories. If a directory with the same timestamp already exists, a numeric suffix is appended (e.g., `_01`, `_02`). Each complete antenna rotation produces one CSV file and one PNG file, written automatically by the echo listener thread. When recording is stopped, a `settings.json` snapshot is written to the recording directory.

## 4.2 Output Directory Structure

Each recording session produces the following directory structure.

```
recordings/<timestamp>/
  csv/rotation_<ts>.csv
  png/rotation_<ts>.png
  settings.json
```

The top-level directory is named with a compact timestamp (e.g., `20260307_143022`). For pattern-based recordings, the sanitized pattern label is appended (e.g., `20260307_143022_sweep1`). The `csv/` and `png/` subdirectories are created on demand when the first rotation is written.

## 4.3 CSV Format

Each CSV file represents one complete antenna rotation. The file begins with a metadata header of `#`-prefixed comment lines that capture all radar settings at the time of recording.

```
# range_index,4
# gain_auto,false
# gain_value,50
# pulse_width,0
# sea_clutter,0
# rain_clutter,0
# rezboost,0
# interference_rejection,false
# bird_mode,0
# doppler,0
# antenna_speed,0
# main_bang,0
# near_stc,0
# middle_stc,0
# far_stc,0
Status  Scale   Range   Gain    Angle   EchoValues
0   4   4   19  0   12,14,8,...
0   4   4   19  1   10,11,9,...
```

After the header, the column line and data rows are tab-separated. Each row represents one spoke and contains a status field (always 0), the range scale index, the range index, the gain percentage (mapped from the 0–255 wire range to 0–100), the azimuth angle in ticks (0–8191), and the echo bin values as a comma-separated list of integers.

### 4.4 Settings Snapshot

When a recording session ends, the server writes a `settings.json` file to the recording directory. The snapshot format depends on how the recording was initiated.

For manual recordings, the snapshot contains the mode (`"manual"`), the recording directory path, the full `CurrentSettings` object, and a `history` array of `SettingChange` entries. Each history entry records a timestamp, the setting name, and the new value, capturing every parameter change made during the recording session.

For pattern-based recordings, the snapshot additionally includes the `PatternInfo` object describing the pattern that was executing, including all pattern steps with their parameter values and repeat counts.

# 5 Pattern System

## 5.1 What Patterns Do

Patterns automate sequences of radar parameter changes. A pattern consists of one or more *steps*, each specifying a set of radar settings and a *repeat count* that determines how many full antenna rotations that step is held before advancing to the next. When recording is active during pattern execution, each rotation produces a CSV and PNG file, allowing systematic data collection across parameter combinations.

## 5.2 Simple Pattern Format (7-Field)

The simple format uses seven comma-separated fields per step. All fields are required and all values are set explicitly.

```
range, gain, pulse, rezboost, speed, interp, repeat
```

For example, the step `4,50,0,3,2,1,5` sets range index 4 (1 NM), gain 50, pulse S1, RezBoost high, auto antenna speed, interpolation on, and holds for 5 rotations. Multiple steps are separated by semicolons.

```
4,50,0,3,2,1,5; 8,50,2,3,2,1,5; 11,50,4,3,2,1,5
```

## 5.3 Advanced Pattern Format (13-Field)

The advanced format extends the simple format with six additional fields for fine-grained signal processing control.

```
range, gain, pulse, rezboost, speed, interp,
    sea_clutter, rain_clutter, main_bang,
    interference_rejection, bird_mode, doppler, repeat
```

Any of the 12 parameter fields (all except `repeat`) can be set to `NaN` to leave that setting unchanged from its current value. This is particularly useful when only a subset of parameters needs to vary across steps. For example, the step `4,NaN,0,NaN,NaN,NaN,NaN,NaN,NaN,NaN,NaN,NaN,3` changes only the range to index 4 and the pulse to S1, leaving all other settings at their current values and holding for 3 rotations.

## 5.4 Pattern Queue

Multiple patterns can be loaded into a queue and executed sequentially. Each queue entry has its own pattern data, repeat count, and optional label. The repeat count controls how many times the entire pattern is cycled ($-1$ for infinite looping). Patterns execute in the order they were added.

During queue execution, each pattern item creates its own recording session with a separate timestamped directory (tagged with the sanitized pattern label if one was provided). The queue can

be stopped at any time using the STOP button, which halts execution between steps. A RESET button clears the entire queue.

### 5.5 Loading Patterns

Patterns can be loaded in two ways. The first is file upload: clicking the "Load file" button opens a file picker that accepts `.txt` or `.csv` files. The file contents are parsed as semicolon-separated steps. The second is manual input: on the settings page, clicking "Manual input" reveals a text area where steps can be typed or pasted directly.

After loading, the user sets a repeat count and an optional label, then clicks "Add to queue" to append the pattern to the queue list. Multiple patterns can be queued before pressing RUN.

### 5.6 NaN Override Behavior

When a pattern step uses `NaN` for a field, that setting remains *unlocked* during execution. This means the user can manually change that parameter through the web UI while the pattern runs, and the change takes effect immediately. Conversely, parameters with explicit values in the current step are *locked*—manual changes to locked parameters are rejected with an error message indicating that the setting is controlled by the active pattern.

This mechanism allows hybrid operation where some parameters are swept by the pattern while others are adjusted interactively. For example, a pattern that sweeps range and pulse but sets all clutter fields to `NaN` lets the operator tune clutter rejection in real time while the automated sweep runs.

# Part II

# Software Internals

## 6 System Architecture Overview

### 6.1 Module Summary

### 6.2 System-Level Block Diagram

### 6.3 Module-Level Diagram

### 6.4 Data Flow Diagram

### 6.5 Concurrency Model

# 7 PPI Rendering

## 7.1 Coordinate Transformation

## 7.2 Color Mapping

## 7.3 Overlay Elements

## 7.4 PNG Encoding

## 7.5 How to Modify

# 8 Web Server

## 8.1 Framework

## 8.2 Shared State

## 8.3 Route Registration

## 8.4 Handler Pattern

## 8.5 Echo Listener Thread

## 8.6 Static File Serving

## 8.7 How to Add a New Endpoint

# 9 REST API Reference

## 9.1 Power and Connection

## 9.2 Settings

## 9.3 State Queries

## 9.4 Echo Image

## 9.5 Recording

## 9.6 Pattern Execution

## 9.7 Stub Endpoints

# 10 Recording System Internals

## 10.1 Echo Listener Integration

## 10.2 Directory Creation

## 10.3 CSV Writing

## 10.4 Settings Snapshots

## 10.5 Setting Change History

# 11 Pattern Execution Engine

## 11.1 Step Parsing

## 11.2 Differential Command Generation

## 11.3 Single Pattern Execution

## 11.4 Queue Execution

## 11.5 NaN Override Logic

## 11.6 Stop Mechanism

# 12 CLI Modes (All Other Modes)

## 12.1 Handshake

## 12.2 TX On / TX Off

## 12.3 Full Cycle

## 12.4 Render Live

## 12.5 Echo Listen

## 12.6 Command Test

## 12.7 Dump Raw

## 12.8 Test Commands

## 12.9 Validate

# 13 Connection State Machine

## 13.1 States and Transitions

## 13.2 RadarError Enum

## 13.3 Timeout Values

## 13.4 Graceful Degradation

## 13.5 Keepalive Handling

# 14 Testing & Validation

## 14.1 Running Tests

## 14.2 Test Categories

### 14.2.1 Connection Tests

### 14.2.2 TX Tests

### 14.2.3 Settings Tests

### 14.2.4 Echo Impact Tests

### 14.2.5 Render Integration Test

## 14.3 A/B Command Testing

## 14.4 Validation Suite

# Part III

# Echo Data Pipeline

## 15 Echo Data Format

### 15.1 Packet Structure

### 15.2 Header Field Breakdown

### 15.3 Echo Metadata Extraction

### 15.4 Spoke Sub-Header

### 15.5 Multi-Spoke Packing

### 15.6 Azimuth and Heading

### 15.7 Typical Packet Characteristics

# 16 Echo Encoding Types

## 16.1 Encoding 0: Raw Uncompressed

## 16.2 Encoding 1: RLE with Repeat Marker

## 16.3 Encoding 2: Delta RLE with Previous Spoke Reference

## 16.4 Encoding 3: 2-Bit RLE with Dual Reference

## 16.5 Encoding Selection

## 16.6 Display Value Conversion

## 16.7 Rotation Detection

# 17 Doppler Processing

## 17.1 Status Field Encoding

## 17.2 Echo Byte Encoding

## 17.3 Downstream Usage

## 17.4 Practical Notes

# 18 Dual-Pulse Interleaving

## 18.1 Pulse Types

## 18.2 Range Dependence

## 18.3 Rez Boost Effect

## 18.4 Spoke Handling

# Part IV

# Network & Wire Protocol

## 19 Network Architecture

### 19.1 Topology

### 19.2 Port Map

### 19.3 Auto-Discovery and Network Configuration

#### 19.3.1 Platform-Specific IP Alias

# 20 Control Protocol

## 20.1 Message Format

## 20.2 Binary Keepalive

## 20.3 Binary vs ASCII Disambiguation

## 20.4 Response Types

## 20.5 Response Parsing Pipeline

## 20.6 Command Acknowledgment

## 20.7 Control Channel Drain

## 20.8 Initialization Sequence

### 20.8.1 Phase 1: Discovery

### 20.8.2 Phase 2: Authentication

### 20.8.3 Phase 3: Control Channel

### 20.8.4 Phase 4: Firmware Query

### 20.8.5 Phase 5: Register Dump

### 20.8.6 Phase 6: Steady State

# 21 Command Reference

## 21.1 TX State ($S69)

## 21.2 Range ($S62)

## 21.3 Gain ($S63)

## 21.4 Sea Clutter ($S64)

## 21.5 Rain Clutter ($S65)

## 21.6 Pulse Width ($S68)

## 21.7 STC Near ($S85)

## 21.8 STC Middle ($S86)

## 21.9 STC Far ($S87)

## 21.10 Main Bang Suppression ($S83)

## 21.11 Antenna Height ($S84)

## 21.12 Antenna Revolution ($S89)

## 21.13 Interference Rejection ($S67, param[2]=2)

## 21.14 Noise Reduction ($S67, param[2]=3)

## 21.15 Rez Boost ($SEE)

## 21.16 Bird Mode ($SED)

## 21.17 Doppler / Target Analyzer ($SEF)

## 21.18 No-Transmit Sectors ($S77)

## 21.19 Query Commands

# 22 Periodic Telemetry

## 22.1 Telemetry Message Table

## 22.2 Telemetry Processing

## 22.3 Parsed vs Discarded Data

# 23 Proprietary Library Reference

## 23.1 Module Breakdown

## 23.2 Shared Command IDs